SUDOKU GAME USING PYTHON TKINTER

AIM

To design and develop an interactive Sudoku Game using Python Tkinter. The player fills a 9×9 grid so that every row, every column, and every 3×3 box contains the numbers 1 to 9 without repetition. The application provides a graphical board, a Check button, a New Game button, and an Exit option.

PROCEDURE

1. Import the tkinter library to create the graphical user interface.
2. Define a 9×9 Sudoku puzzle, where 0 represents an empty cell.
3. Create the main window and display the game title and instructions.
4. Use Entry widgets to create the 9×9 Sudoku grid.
5. Display the given puzzle numbers as read-only values so they cannot be changed.
6. Allow the player to select empty cells and enter numbers from 1 to 9.
7. Create CHECK, NEW GAME, and EXIT buttons.
8. When CHECK is clicked, read the values entered in the grid.
9. Validate that all cells contain numbers from 1 to 9.
10. Check rows, columns, and 3×3 boxes for repeated numbers.
11. Display a success message if the board is complete and valid; otherwise show an error hint.
12. Use NEW GAME to reset the puzzle and EXIT to close the application.

PROGRAM CODE

Save the code as **sudoku_game.py** and run it using Python 3. Tkinter is included with most standard Python installations.

```
import tkinter as tk
import random

PUZZLE = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

class SudokuGame:
    def __init__(self, root):
        self.root = root
        self.root.title("SUDOKU GAME")
        self.root.resizable(False, False)
        self.root.configure(bg="#172554")
        self.original = [row[:] for row in PUZZLE]

        tk.Label(root, text="SUDOKU GAME", font=("Arial Black", 20),
                 bg="#172554", fg="white").pack(pady=8)
        tk.Label(root, text="Fill each row, column and 3x3 box with 1-9",
                 font=("Arial", 10), bg="#172554",
                 fg="#BFDBFE").pack(pady=3)

        self.board_frame = tk.Frame(root, bg="#172554")
        self.board_frame.pack(padx=12, pady=8)
        self.cells = []
        self.entries = []

        for r in range(9):
            row_cells = []
            row_entries = []
            for c in range(9):
                cell = tk.Frame(
                    self.board_frame,
                    highlightbackground="#1D4ED8" if r % 3 == 0 or c % 3 == 0
                    else "#94A3B8",
                    highlightthickness=1
                )
                cell.grid(row=r, column=c, padx=(1 if c % 3 else 2),
                          pady=(1 if r % 3 else 2))
                value = self.original[r][c]
                entry = tk.Entry(
                    cell, width=2, font=("Arial", 16, "bold"),
                    justify="center", relief="flat",
                    bg="#DBEAFE" if value else "white",
                    fg="#172554" if value else "#2563EB"
                )
                entry.pack(ipady=5)
                if value:
                    entry.insert(0, str(value))
                    entry.config(state="readonly")
                row_entries.append(entry)
            self.entries.append(row_entries)

        self.message = tk.Label(
            root, text="Select an empty cell and type 1-9.",
            font=("Arial", 10, "bold"), bg="#172554", fg="#86EFAC"
        )
        self.message.pack(pady=4)

        buttons = tk.Frame(root, bg="#172554")
        buttons.pack(pady=8)
        tk.Button(buttons, text="CHECK", font=("Arial", 10, "bold"),
                  bg="#22C55E", command=self.check_board).pack(side="left", padx=7)
        tk.Button(buttons, text="NEW GAME", font=("Arial", 10, "bold"),
                  bg="#38BDF8", command=self.new_game).pack(side="left", padx=7)
        tk.Button(buttons, text="EXIT", font=("Arial", 10, "bold"),
                  bg="#EF4444", fg="white",
                  command=root.destroy).pack(side="left", padx=7)

    def read_board(self):
        board = []
        for row in self.entries:
            values = []
            for entry in row:
                value = entry.get().strip()
                values.append(int(value) if value.isdigit() else 0)
            board.append(values)
        return board

    def check_board(self):
        board = self.read_board()
        for r in range(9):
            for c in range(9):
                value = board[r][c]
                if value < 1 or value > 9:
                    self.message.config(
                        text="Keep filling the empty cells with numbers 1-9.",
                        fg="#FACC15")
```

```
                    return

                if self.original[r][c] == 0:
                    for k in range(9):
                        if k != c and board[r][k] == value:
                            self.message.config(
                                text=f"Repeated number in row {r + 1}.",
                                fg="#FCA5A5")
                            return
                        if k != r and board[k][c] == value:
                            self.message.config(
                                text=f"Repeated number in column {c + 1}.",
                                fg="#FCA5A5")
                            return

                    box_r, box_c = (r // 3) * 3, (c // 3) * 3
                    for rr in range(box_r, box_r + 3):
                        for cc in range(box_c, box_c + 3):
                            if (rr, cc) != (r, c) and board[rr][cc] == value:
                                self.message.config(
                                    text="Repeated number in a 3x3 box.",
                                    fg="#FCA5A5")
                                return

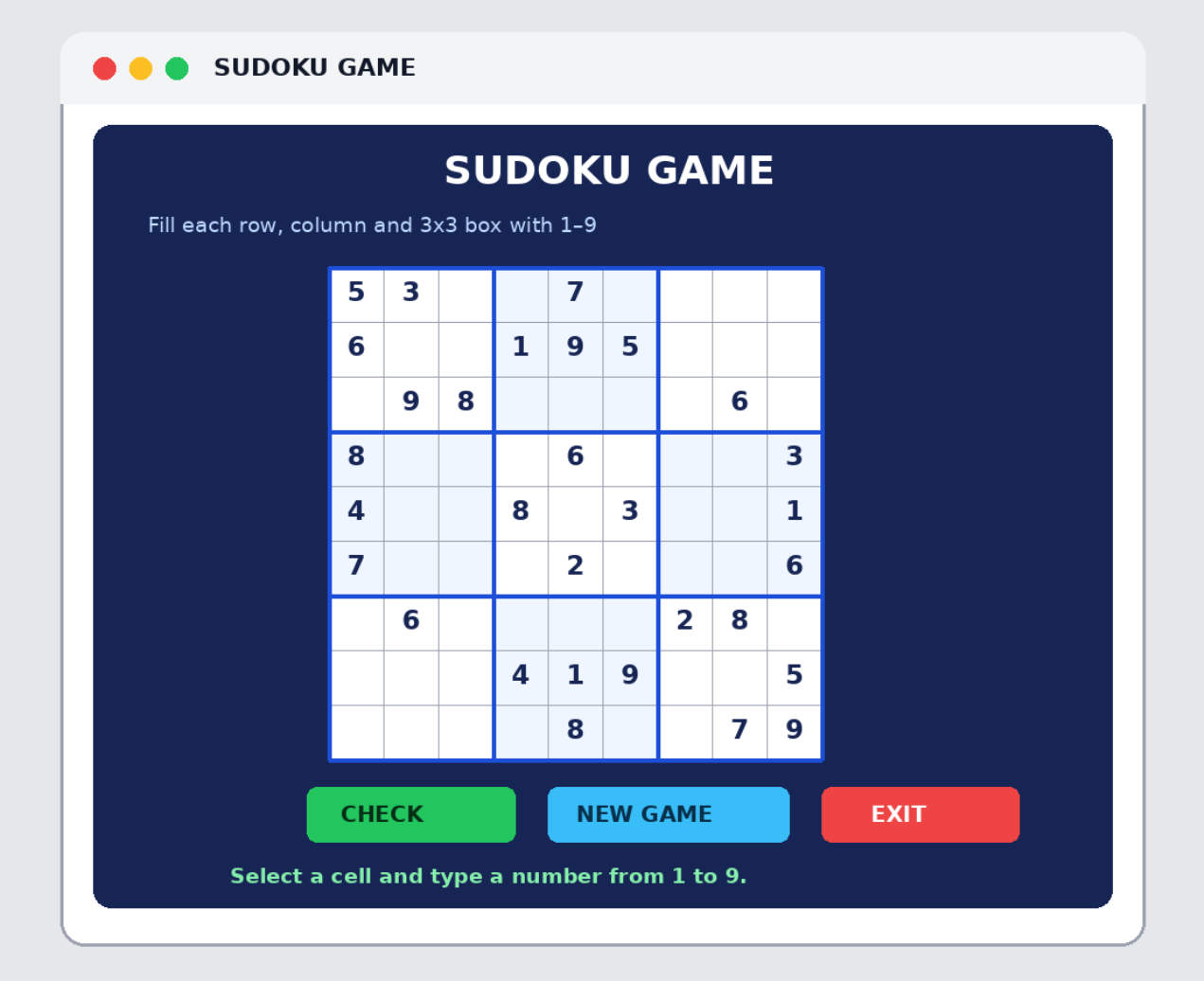
        self.message.config(text="Congratulations! Sudoku completed correctly!",
                            fg="#86EFAC")

    def new_game(self):
        for r in range(9):
            for c in range(9):
                entry = self.entries[r][c]
                entry.config(state="normal")
                entry.delete(0, tk.END)
                value = self.original[r][c]
                if value:
                    entry.insert(0, str(value))
                    entry.config(state="readonly")
        self.message.config(text="New game ready. Fill the empty cells.",
                            fg="#86EFAC")


if __name__ == "__main__":
    root = tk.Tk()
    game = SudokuGame(root)
    root.mainloop()
```

OUTPUT

The picture below illustrates the Sudoku game window with a 9×9 grid, fixed starting numbers, and CHECK, NEW GAME, and EXIT buttons.



RESULT

Thus, a Sudoku Game interface was developed using Python Tkinter. It demonstrates GUI design, grid creation, input handling, nested loops, conditional statements, and validation of Sudoku rows, columns, and 3×3 boxes.